SOMAIYA VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering

Department of Electronics and Telecommunication Engineering
Programme: Computer and Communication Engineering
LY B.Tech – CCE – July to November 2026

DEVOPS INTERNAL ASSESSMENT-II

SecureMonitor

Automated DevSecOps Security Scanning, CI Gate Enforcements, and Real-Time Operational Application Monitoring

Task / Activity:	Internal Assessment-II (20 Marks)
Student Name(s):	Aditi Gupta & Chinmay Parate
Roll Number(s):	16014123020 & 16014123018
Academic Year:	LY B.Tech CCE (Sem VII, July–Nov 2026)
Repository:	https://github.com/Aditi141004/SecureMonitor

1. BRIEF DESCRIPTION OF THE TASK

SecureMonitor is a Python Flask web application designed to demonstrate an integrated DevSecOps lifecycle and real-time operational observability framework. The project establishes an automated Continuous Integration (CI) pipeline using GitHub Actions that executes automated unit tests and enforces a strict dependency security gate before code changes are validated.

Static code analysis is performed via GitHub CodeQL to detect potential implementation vulnerabilities, while dynamic application security testing (DAST) is executed locally using OWASP ZAP against the live web application. Dynamic response security headers were implemented to mitigate web exposure risks. For post-deployment operational observability, Prometheus scrapes system metrics from a dedicated metric endpoint, and Grafana visualizes real-time application throughput, response latency, failure counts, and health status.

2. PLATFORMS AND TOOLS USED

Platform / Tool	Role & Specific Application Purpose
Python / Flask	Core web application framework exposing operational endpoints (`/`, `/health`, `/api/data`, `/api/error`, and `/metrics`).
pytest	Automated unit testing framework validating endpoint HTTP status codes, JSON response structures, and metric exports.
Git & GitHub	Distributed version control system and repository hosting platform for code management and trigger-based workflow orchestration.
GitHub Actions	CI/CD engine running automated test suites, building code artifacts, and executing security gates on repository events.
pip-audit	Dependency vulnerability scanner integrated into the CI workflow to strictly audit Python package dependencies.
CodeQL	Semantic code analysis engine executing static application security testing (SAST) for Python and GitHub Actions workflows.
OWASP ZAP	Dynamic Application Security Testing (DAST) containerized tool running passive baseline scans against the local application.
Prometheus	Time-series monitoring server scraping multi-dimensional metrics from the application's `/metrics` endpoint.
Grafana	Visualization platform connected to Prometheus as a data source to render operational dashboards and health indicators.
Docker Desktop	Containerization platform running isolated containerized environments for Prometheus and OWASP ZAP.

3. STEP-BY-STEP WORKFLOW

1. **Application Development & Instrumentation:** Developed a Flask web application exposing `/health` and `/metrics` endpoints instrumented with Prometheus client metrics including `http_requests_total`, `http_request_duration_seconds`, `http_requests_failed_total`, and `app_health`.
2. **Automated Unit Testing:** Implemented unit tests using `pytest` to verify endpoint connectivity, JSON response schema consistency, error handling, and metric generation.
3. **CI Pipeline Setup & Security Gate Enforcement:** Configured a GitHub Actions workflow that executes unit tests and enforces a strict dependency vulnerability audit using `pip-audit`.
4. **Code Scanning & Web Vulnerability Analysis:** Executed GitHub CodeQL workflows for static analysis and executed OWASP ZAP baseline scanning against the local Flask application. Applied security response headers to harden the application.
5. **Metrics Collection & Dashboard Visualization:** Configured Prometheus to scrape the Flask application metrics and constructed a multi-panel Grafana dashboard to track performance and system state.

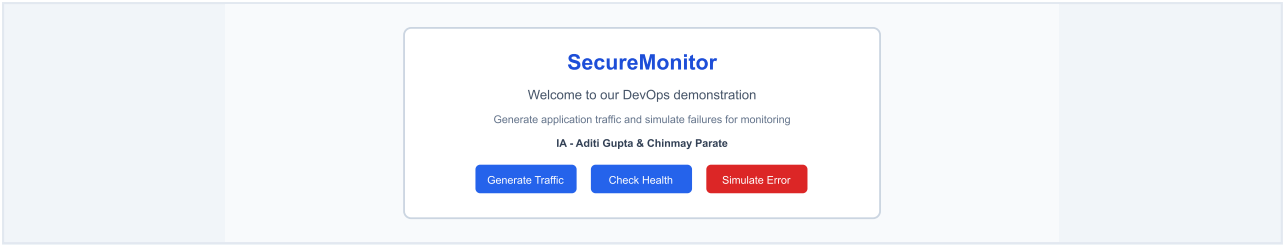


Figure 1: SecureMonitor Web Application Interface. The Flask landing page hosted locally at `127.0.0.1:5000` displaying application controls for traffic generation, health verification, and error simulation.

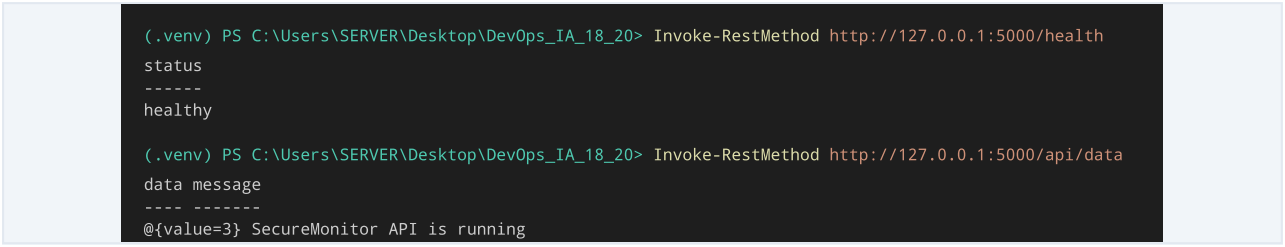
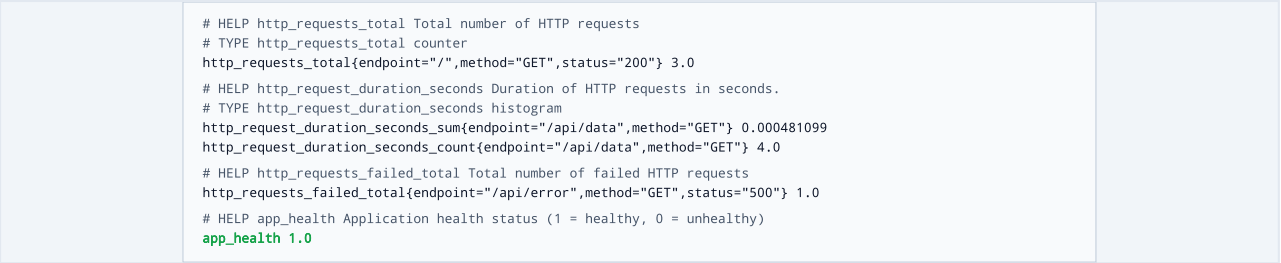


Figure 2: Endpoint Health & Data Route Verification. Terminal REST invocation verifying that `/health` returns status `healthy` and `/api/data` responds with valid JSON operational payloads.

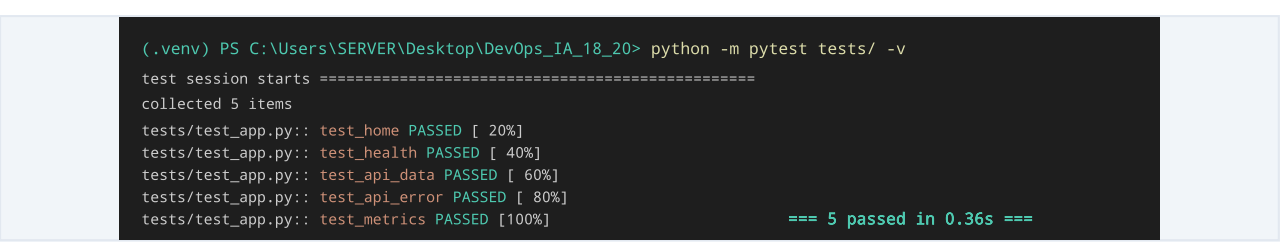
4. APPLICATION INSTRUMENTATION & AUTOMATED TESTING

The Flask application was instrumented using the Prometheus Python client library to expose real-time metrics at `/metrics`. The exported metrics include custom counters for HTTP request totals, failure counts by status code, request duration histograms, and application health gauges. Automated unit tests were written in `pytest` to validate all functional routes and confirm the presence of required metric keys.



```
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{endpoint="/",method="GET",status="200"} 3.0
# HELP http_request_duration_seconds Duration of HTTP requests in seconds.
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_sum{endpoint="/api/data",method="GET"} 0.000481099
http_request_duration_seconds_count{endpoint="/api/data",method="GET"} 4.0
# HELP http_requests_failed_total Total number of failed HTTP requests
http_requests_failed_total{endpoint="/api/error",method="GET",status="500"} 1.0
# HELP app_health Application health status (1 = healthy, 0 = unhealthy)
app_health 1.0
```

Figure 3: Prometheus Metrics Export Verification. Inspection of `127.0.0.1:5000/metrics` confirming active instrumentation for `http_requests_total`, request duration histograms, error counters (500 status), and `app_health` set to 1.0.



```
(.venv) PS C:\Users\SERVER\Desktop\DevOps_IA_18_20> python -m pytest tests/ -v
test session starts ============================================
collected 5 items
tests/test_app.py:: test_home PASSED [ 20%]
tests/test_app.py:: test_health PASSED [ 40%]
tests/test_app.py:: test_api_data PASSED [ 60%]
tests/test_app.py:: test_api_error PASSED [ 80%]
tests/test_app.py:: test_metrics PASSED [100%]                    === 5 passed in 0.36s ===
```

Figure 4: Automated Unit Test Suite Execution. Local execution of `pytest` demonstrating 5 passed unit tests covering all routes, error simulation, and metric reporting in 0.36 seconds.

5. CI AUTOMATION & DEPENDENCY SECURITY GATE

Continuous Integration is configured via GitHub Actions in `.github/workflows/ci.yml`. The pipeline defines two dependent jobs: `test` and `security-gate`. The `test` job sets up Python, installs dependencies, and runs `pytest`. Upon successful completion of unit tests, the `security-gate` job executes `pip-audit` in strict mode (`python -m pip_audit -r app/requirements.txt --strict`) against declared dependencies.

If any vulnerable package is detected, the security gate fails, causing the entire workflow to fail and preventing downstream integration. In the recorded run for commit `dfb4f12`, both jobs completed successfully.

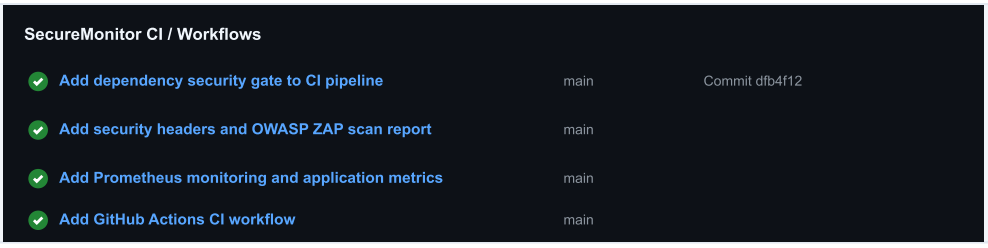


Figure 5: GitHub Actions CI Workflow Execution History. Dashboard view displaying successful completion across all CI workflow runs, culminating in the integration of the dependency security gate.

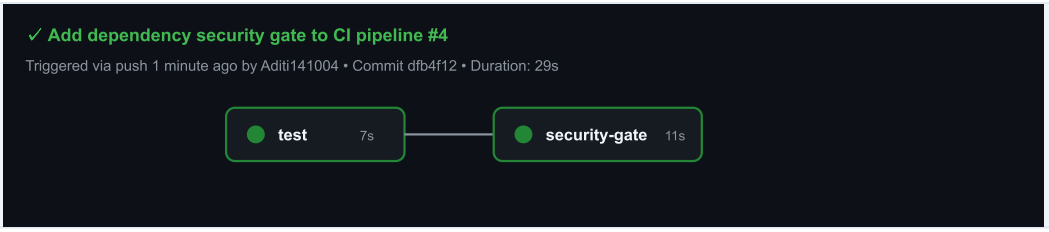


Figure 6: DevSecOps Security Gate Pipeline Execution. Detail view of GitHub Actions run for commit `dfb4f12` showing sequential execution and success of both the `test` and `security-gate` jobs.

6. CODEQL & OWASP ZAP SECURITY SCANNING

Static Application Security Testing (SAST) was performed using GitHub CodeQL to analyze source code for structural security risks across matrix jobs (Python and GitHub Actions). For Dynamic Application Security Testing (DAST), OWASP ZAP was executed in a baseline container against the running Flask instance.

To mitigate baseline security findings, custom security headers (including `X-Content-Type-Options` and Content Security Policy rules) were injected via an `after_request` handler in Flask. A subsequent ZAP scan confirmed **62 PASS checks** and **0 FAIL alerts**, leaving 5 informational warning types (concerning server version disclosure, cache control, CSP directives, COOP headers, and application detection) requiring ongoing review.

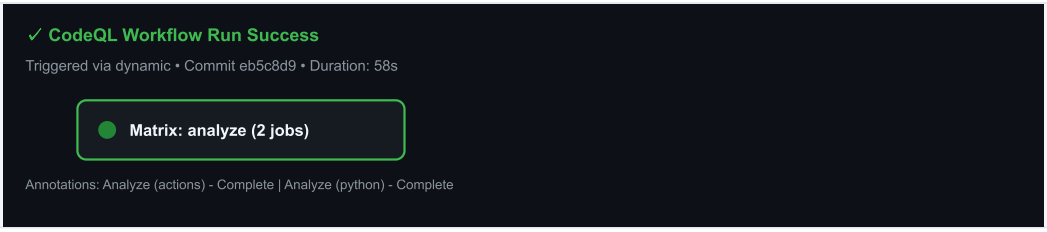


Figure 7: GitHub CodeQL Static Analysis Execution. Workflow run summary showing successful execution of CodeQL security analysis matrix jobs for Python and GitHub Actions.

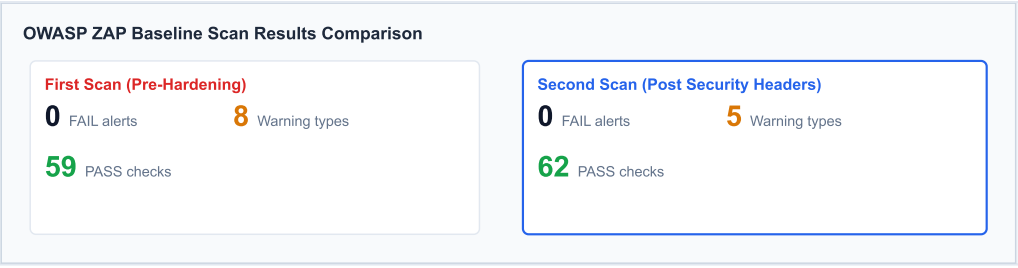


Figure 8: OWASP ZAP Dynamic Scan Result Progress. Comparison demonstrating the reduction of ZAP warning types from 8 to 5 and increase of PASS checks to 62 following security response header implementation.

7. OPERATIONAL MONITORING WITH PROMETHEUS

Prometheus was deployed via Docker Desktop to scrape metrics from the Flask application running on the host machine using the `host.docker.internal:5000` bridge interface. The `prometheus.yml` configuration file was set up with a scrape target named `secure-monitor`.

Target state verification on the Prometheus web console (`http://localhost:9090/targets`) confirmed that the scrape endpoint was active and reported state **UP (1/1)**. PromQL queries executed in Prometheus successfully retrieved time-series data for total HTTP requests and application health metrics.



Figure 9: Prometheus Target Health Verification. Target dashboard at `localhost:9090/targets` verifying that the Flask application scrape endpoint is connected and reporting an UP health state.



Figure 10: PromQL Metric Expression Queries. Execution of PromQL queries in Prometheus demonstrating successful retrieval of `http_requests_total` time-series vectors and `app_health` gauge values.

8. REAL-TIME VISUALIZATION WITH GRAFANA

Grafana was configured with Prometheus as its primary time-series data source. A customized operational dashboard was developed to monitor system performance and health across six dedicated visualization panels: Application Status, Request Rate (req/sec), Error Rate (%), Response Latency, Total Request Volume, and Security Status.

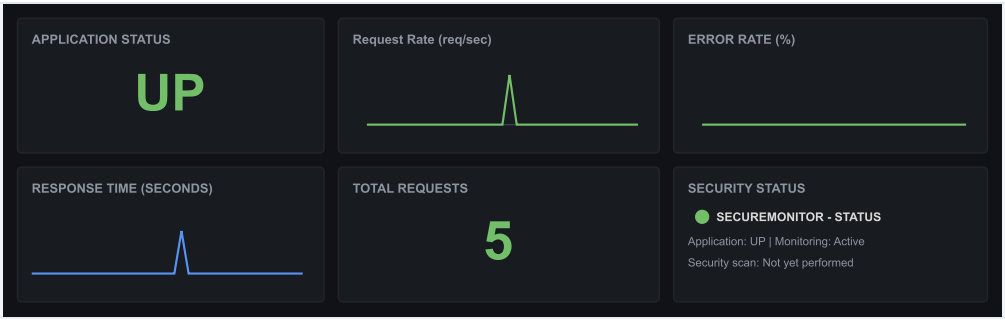


Figure 11: SecureMonitor Grafana Operational Dashboard. Live dashboard displaying application availability (UP), request throughput, 0% error rate, request latencies, request counter (5 total), and security monitoring status.

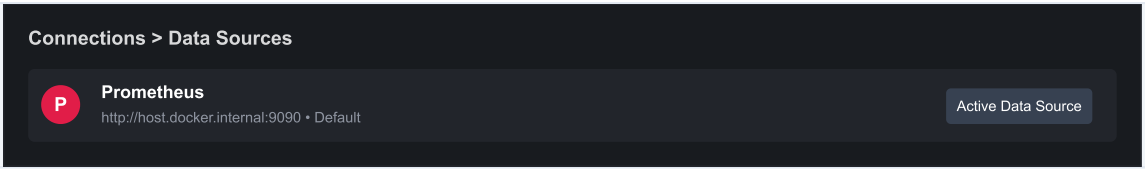


Figure 12: Prometheus Data Source Integration in Grafana. Configuration interface at `localhost:3000` confirming successful connection between Grafana and the Prometheus time-series backend.

9. SUMMARY OF IMPLEMENTATION RESULTS

Verification Phase	Observed Result & Factual Evidence
Automated Unit Tests	5 of 5 tests passed in 0.36 seconds using `pytest` (covering home, health, api_data, api_error, metrics).
Dependency Audit	Executed `pip-audit` strictly; output confirmed `No known vulnerabilities found` at the time of check.
GitHub Actions CI Gate	Commit `dfb4f12` successfully executed both `test` and `security-gate` jobs.
Code Scanning (SAST)	GitHub CodeQL workflow completed successfully for Python and Actions analysis matrices.
OWASP ZAP Scan (DAST)	Dynamic baseline scan reported **62 PASS checks**, **0 FAIL alerts**, and 5 warning types post-header injection.
Prometheus Metrics	Target verified as `UP (1/1)` at `host.docker.internal:5000/metrics`; PromQL queries fetched active vectors.
Grafana Dashboard	Successfully rendered real-time application uptime, request throughput, latencies, and security panel status.

10. CONCLUSION & KEY TAKEAWAYS

The **SecureMonitor** project successfully demonstrates an end-to-end integration of automated DevOps CI/CD pipelines, DevSecOps security gating, and real-time operational observability. By combining shift-left dependency auditing with dynamic application security testing and time-series metrics visualization, the implementation achieves robust quality assurance and operational transparency.

Key Engineering Learnings:

- **Shift-Left Security Enforcement:** Integrating `pip-audit` as a blocking security gate in GitHub Actions ensures that vulnerable third-party dependencies are intercepted prior to code integration.
- **Defense-in-Depth Security Analysis:** Combining static code analysis (CodeQL) with dynamic vulnerability scanning (OWASP ZAP) provides comprehensive coverage across both code structure and runtime HTTP behavior.
- **Operational Observability:** Exposing Prometheus-compatible metrics allows engineering teams to monitor real-time application throughput, request latencies, and failure spikes via unified Grafana dashboards.

Project Repository: https://github.com/Aditi141004/SecureMonitor
Report Authors: Aditi Gupta (16014123020) & Chinmay Parate (16014123018)
Course / Assessment: DevOps - Internal Assessment-II (LY B.Tech CCE, July–Nov 2026)